

Лекция 5

Эмбединги и векторные базы
как искать нужное по смыслу, а не по словам

- ❶ **Проблема:** поиск по словам ломается на синонимах
- ❷ **Эмбединги:** текст как вектор смысла
- ❸ **Близость:** как её меряют и как ищут топ-k
- ❹ **Векторные базы:** FAISS, Chroma, Qdrant и когда что

Куда это ведёт

На прошлой лекции третьим способом памяти был «достать нужное заново». Сегодня разберём его сердце: поиск по смыслу. На следующей соберём из него RAG.

- Запрос «где мой заказ» и документ «как отследить статус посылки» про одно, но **ни одного общего слова**
- Поиск по ключевым словам их не свяжет: он ищет совпадение букв, а не смысла
- Синонимы, перефразировки, другой язык, опечатки: всё это ломает поиск по словам
- А пользователь спрашивает своими словами, не словами из документа

Что нужно

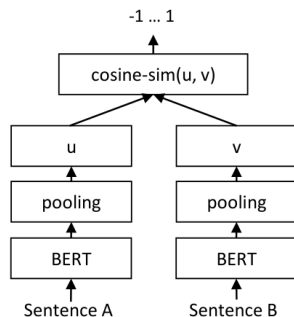
Способ сказать, что две фразы **про одно и то же**, даже если слова разные. Это и делают эмбединги: превращают смысл в числа, которые можно сравнивать.

- Модель-эмбеддер превращает любой текст в **вектор** из сотен или тысяч чисел
- Это координаты точки в многомерном пространстве. Близкие по смыслу тексты оказываются **рядом**, далёкие по смыслу далеко
- Числа сами по себе не читаются, важны только расстояния между ними
- Обучен эмбеддер так, что «где мой заказ» и «отследить посылку» встают близко, хотя слова разные

Ключевая мысль

Смысл становится геометрией. Вопрос «похожи ли два текста» превращается в «близко ли две точки», а это уже арифметика.

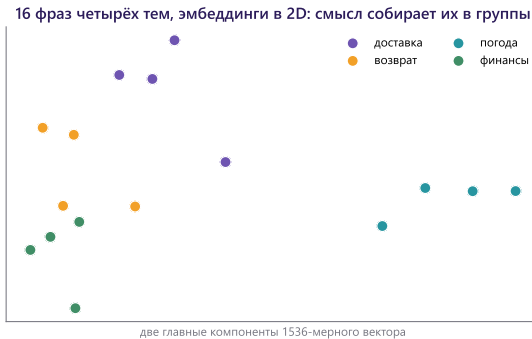
Откуда берётся вектор: устройство эмбеддера



Источник: Reimers, Gurevych, 2019. Sentence-BERT

- Внутри тот же трансформер, что в чат-модели, но без генерации: текст проходит через слои, векторы токенов **усредняются** в один
- Учат на парах: похожие по смыслу тексты тянут друг к другу, разные разводят. Миллионы пар вопросов и ответов, перефразировок, переводов
- Поэтому «где мой заказ» и «отследить посылку» встают рядом: модель видела тысячи таких пар
- Чат-модель и эмбеддер это разные модели. Чат-модель для эмбеддингов не используют, а эмбеддер не умеет отвечать

Как это выглядит: реальная карта смыслов



- Фразы прогнали через эмбеддер и сжали 1536 измерений в две главные компоненты
- Модель никто не учил темам, но фразы **сами собрались в группы** по смыслу
- Где группы соприкасаются, там смысл и правда близок: «вернуть деньги за подписку» тянется от возврата к финансам

- Меряют не расстояние, а **угол** между векторами: косинусную близость
- Смотрят в одну сторону: близость около 1, тексты про одно. Перпендикулярны: около 0, не связаны
- Почему угол, а не длина: длина вектора зависит от размера текста, а направление от смысла
- На практике векторы **нормализуют** к единичной длине, и тогда косинус это просто скалярное произведение

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

После нормализации

$$\hat{a} \cdot \hat{b}$$

Одно умножение матрицы на вектор считает близость запроса ко всему корпусу разом. Быстро и просто.

Поиск в действии: близость к запросу



- Один запрос, шесть документов, для каждого посчитана косинусная близость
- Верные строки про заказ и доставку всплывают наверх, погода и оплата остаются внизу, **хотя слова у запроса и ответа разные**
- Забрать топ-к самых близких и есть поиск по смыслу. Дальше их отдают модели

Модели эмбеддингов: чем считать векторы

Вариант	Плюсы	Минусы
sentence-transformers OpenAI, Cohere (API) Мультиязычные модели	бесплатно, локально, приватно сильные, ничего не хостить русский и английский рядом	нужен свой compute, слабее топовых платно, данные уходят наружу чуть слабее моноязычных



sentence-
transformers



OpenAI



Cohere



Voyage



Jina



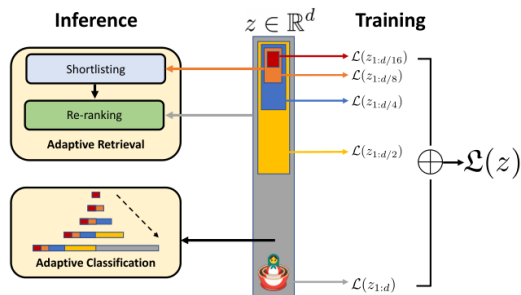
BGE



Mistral

- **Размерность** вектора: 384, 768, 1536. Больше не всегда лучше, но всегда дороже по памяти
- На курсе локальный sentence-transformers для приватности и цены, API когда нужно сильнее. Сравнивать модели помогает таблица MTEB

Размерность можно резать: эмбединги-матрёшки



Источник: Kusupati et al., 2022. Matryoshka Representation Learning

- Обычный вектор нельзя укоротить: смысл размазан по всем координатам поровну
- Матрёшку учат так, что **первые координаты самые важные**: первые 256–512 из 1536 сохраняют почти всё качество
- Так устроены современные API-эмбеддеры: параметр `dimensions` в запросе, и вектор короче в несколько раз
- Зачем: хранить и искать по коротким векторам дешево и быстро, а полный вектор достать, когда нужна точность

```
# локально, бесплатно
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("intfloat/multilingual-e5-base")
vectors = model.encode(chunks, normalize_embeddings=True) # (N, 768)

# или через API
import requests
r = requests.post("https://api.openai.com/v1/embeddings",
    headers={"Authorization": f"Bearer {key}"},
    json={"model": "text-embedding-3-small", "input": chunks})
vectors = [row["embedding"] for row in r.json()["data"]] # (N, 1536)
```

- На вход список текстов, на выходе матрица векторов. Одна модель на весь корпус и на запросы
- **Нормализуйте сразу:** тогда близость это просто скалярное произведение

- Эмбединг текста не меняется, пока не меняется текст и модель. Значит, считать его повторно бессмысленно
- Корпус эмбеждят один раз при сборке индекса. Запрос эмбеждят на лету, он короткий
- **Кэш** по хэшу текста: видели строку, берём готовый вектор. Так демо-агент курса не платит дважды за одно
- Пересчёт нужен только при смене модели эмбеждера или изменении текста

Порядок цен

Эмбединги дешёвы: миллион токенов стоит центы. Дорого не считать, а **хранить и искать** по миллионам векторов. Отсюда векторные базы.

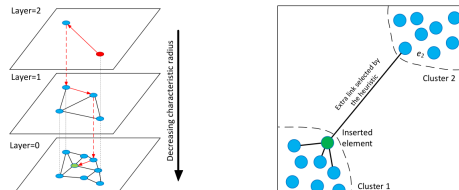
```
import numpy as np
# vectors: (N, d) уже нормализованы
q = embed([query])[0]          # (d,)
sims = vectors @ q              # (N,) косинусы
top = np.argsort(-sims)[:k]     # индексы топ-k
chunks_found = [chunks[i] for i in top]
```

Пять строк. Одно умножение матрицы на вектор даёт близость ко всему корпусу.

- Для тысяч и даже десятков тысяч кусков этого **достаточно**. numpy справится за миллисекунды
- Не усложняйте раньше времени: векторная база нужна не всем
- Когда векторов миллионы, перебор каждого запроса становится дорогим. Тогда берут индекс

Когда перебор не тянет: приблизительный поиск

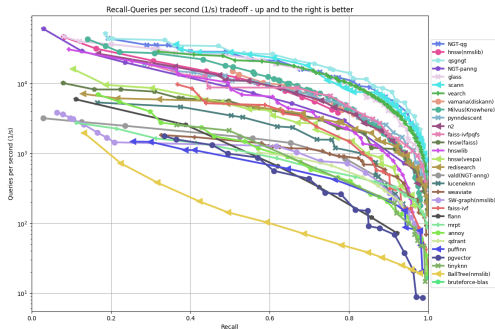
- Перебор честно сравнивает запрос с каждым вектором. На миллионах это медленно
- Векторный индекс (ANN) заранее раскладывает векторы так, чтобы искать соседей, не перебирая всё
- Плата за скорость: поиск становится **приблизительным**, изредка пропускает соседа. Обычно это незаметно
- Индекс окупается на сотнях тысяч векторов и выше. На учебном корпусе из сотен PDF хватит честного перебора



HNSW: слои графа как автострады и улицы.
Верхние переносят далеко, нижние доводят до
точного соседа

Источник: Malkov, Yashunin, 2016

Размен на графике: точность против скорости



Источник: ann-benchmarks.com, датасет glove-100-angular

- По горизонтали recall, доля найденных истинных соседей. По вертикали запросов в секунду, шкала логарифмическая. Правее и выше лучше
- Каждая кривая это один индекс с разными настройками. Честный перебор в правом нижнем углу: recall единица, но в сотни раз медленнее
- Графовые индексы, HNSW и родня, держатся в правом верхнем углу. Поэтому они внутри почти каждой векторной базы

FAISS, Chroma, Qdrant: что выбрать

Инструмент	Что это	Когда брать
numpy	перебор руками	до десятков тысяч векторов
FAISS	библиотека индексов в памяти	много векторов, всё в одном процессе
Chroma	лёгкая база с хранением	прототип, метаданные, персистентность
Qdrant	полноценная векторная БД	прод, фильтры, масштаб, сервер



FAISS



Chroma



Qdrant



Weaviate



Milvus



pgvector



Pinecone



LanceDB



Elastic

- Двигайтесь снизу вверх: начните с numpy или Chroma, усложняйте, когда упрётесь. Выбор базы это **последнее**, о чём стоит думать: сначала качество кусков и эмбеддингов
- Дальше по списку: Weaviate и Milvus как серверные базы, pgvector когда уже есть Postgres, Pinecone как облако, Elastic когда нужен и обычный поиск

- У индекса надо задать, как мерить близость: косинус, скалярное произведение или расстояние L2
- Модель обучена под конкретную меру. Возьмёте не ту, поиск станет тихо **хуже**, но не сломается
- Частый рецепт: нормализовать векторы и мерить скалярным произведением. Это и есть косинус
- В FAISS это IndexFlatIP для нормализованных векторов, а не IndexFlatL2

Тихая поломка

Неверная мера не падает с ошибкой. Индекс работает, находит соседей, просто не тех. Ловится только замером качества поиска, о котором на следующей лекции.

- Вектор нужен для поиска, но агенту в контекст идёт **текст**, а не числа
- Поэтому с каждым вектором хранят: исходный кусок текста, имя документа и позицию, дату, любые метаданные
- Источник обязателен: без него агент не сможет сослаться, а вы не проверите ответ
- Метаданные дают **фильтры**: искать только в свежих документах, только по нужному разделу

Правило

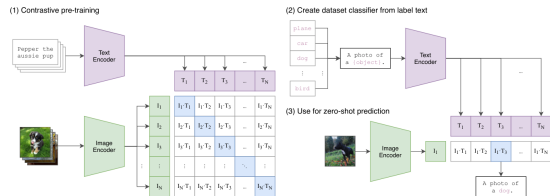
Индекс это не только векторы. Это векторы плюс всё, что понадобится, чтобы показать найденное человеку и подложить модели с указанием, откуда оно.

- Документы добавляются и удаляются. База должна уметь и то, и другое, не пересобираясь целиком
- Смертельная ошибка: индекс собран одной моделью, а запрос считается **другой**
- Векторы разных моделей живут в разных пространствах. Близость между ними бессмысленна
- Версию эмбеддера фиксируют в конфиге. Сменили модель, пересобрали весь индекс, иначе поиск тихо врёт

Из вебинара

В демо это ловили руками: индекс на all-MiniLM, запрос на paraphrase-MiniLM. Размерности совпали, код не упал, а поиск возвращал мусор. Симптом: «RAG работает, но находит ерунду».

- Хороший мультязычный эмбеддер ставит русский вопрос и английский документ про одно и то же **близко**. Корпус на одном языке можно искать вопросами на другом
- Если корпус и вопросы на русском, берите модель, которую учили и на русском. Английская монопольная на кириллице заметно слабее
- Та же идея работает между **текстом и картинкой**: CLIP учит два энкодера класть подпись и её фото в одну точку. Так ищут картинки по словам



Матрица близостей: подпись и её фото на диагонали, всё остальное разводят

Источник: Radford et al., 2021. CLIP

- 1 Индекс одной моделью, запрос другой: поиск возвращает мусор, а код не падает
- 2 Неверная мера близости: L2 там, где нужен косинус. Тихо хуже
- 3 Забыли нормализовать векторы, а меру взяли под нормализованные
- 4 Пересчитывают эмбединги каждый запуск вместо кэша: время и деньги на ветер
- 5 Хранят только вектор без исходного текста и источника: нечего показать и не на что сослаться
- 6 Тянут Qdrant на корпус из ста документов, где хватило бы nupur
- 7 Английская монопольная модель на русском корпусе: качество проседает незаметно

Руками

- Эмбединги корпуса через sentence-transformers
- Наивный поиск на numpy, потом индекс FAISS или Chroma
- Кэш эмбедингов на диске
- Поиск топ-k по своему набору вопросов, взгляд на близости

К следующей лекции

У нас будет поиск по смыслу.
Останется собрать вокруг него RAG:
как резать документы на куски, что
делать, когда поиск нашёл не то, и
как всё это измерить.

- ❶ Поиск по словам ломается на синонимах и перефразировках. Нужен поиск по смыслу
- ❷ Эмбединг превращает текст в вектор, и близость векторов это близость смысла. Смысл становится геометрией
- ❸ Меряют косинусом. Нормализуйте векторы, и близость ко всему корпусу это одно умножение матрицы на вектор
- ❹ Начинайте с перебора на `numpy`. Векторная база нужна на сотнях тысяч векторов, не раньше
- ❺ Одна модель на индекс и на запрос, верная мера близости, рядом с вектором храните текст и источник

Вопросы?

следующая лекция: RAG, который работает. Chunking, гибридный поиск, re-ranking и как это измерить